EXPERIMENT 01

Installation and Configuration of Qiskit

Aim

To install and verify the Python environment required for quantum computing experiments.

Objectives

1. Identify the Python and Jupyter environment.
2. Install Qiskit in an isolated environment.
3. Verify that a basic quantum circuit can be created.

Requirements

Python, terminal/Command Prompt, pip, Jupyter Notebook, Qiskit.

Theory

Qiskit is a Python-based framework used to construct, manipulate, and simulate quantum circuits. A virtual environment isolates project dependencies and reduces conflicts between package versions. A quantum circuit contains qubit wires and a sequence of gates; for n qubits its state is represented by 2^n complex amplitudes. Successful installation is confirmed by importing the framework and constructing a circuit, rather than relying only on a package installation message.

Key Equation(s)

Dimension of n-qubit state space: $\dim(H) = 2^n$.

Algorithm / Steps of Execution

1. Check the installed Python version using python --version.
2. Create a project directory and Python virtual environment.
3. Activate the environment and install Qiskit and Jupyter.
4. Launch Jupyter Notebook and create a Python notebook.
5. Import QuantumCircuit, construct a one-qubit circuit, and display it.
6. Record installed versions and confirm that the import and circuit display work.

Applications

1. Quantum programming education.
2. Reproducible simulation environments.
3. Prototype development for quantum algorithms.

Notebook: EXP-01.ipynb

Aim

EXPERIMENT 02

Quantum Statevectors: Creation, Validation and Visualization

To create quantum statevectors and inspect their normalization and measurement probabilities.

Objectives

1. Represent single-qubit states using complex amplitudes.
2. Test whether a statevector is normalized.
3. Visualize sampled measurement frequencies.

Requirements

Python, Qiskit Statevector, NumPy, Jupyter Notebook, Matplotlib.

Theory

A statevector specifies the amplitudes of all computational basis states. For a single qubit, the amplitudes α and β determine the probabilities of measuring 0 and 1, respectively. A physically valid pure state has unit norm. Qiskit's Statevector class allows vectors to be constructed, displayed, validated and sampled. The observed histogram approximates the theoretical probabilities, with finite-shot variation.

Key Equation(s)

$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$; $|\alpha|^2 + |\beta|^2 = 1$; $P(0) = |\alpha|^2$; $P(1) = |\beta|^2$.

Algorithm / Steps of Execution

1. Import Statevector, NumPy and the histogram plotting utility.
2. Create normalized and non-normalized example vectors.
3. Display the amplitude representation of each vector.
4. Call is_valid() to check normalization.
5. Sample measurements from a valid statevector for a fixed shot count.
6. Plot the histogram and compare counts with expected probabilities.

Applications

1. Quantum state preparation.
2. Validation of simulation inputs.

Aim

3. Measurement probability analysis.

Notebook: EXP-02.ipynb

EXPERIMENT 03

Linear Algebra: Inner Products and Tensor Products

To perform fundamental vector operations used in multi-qubit quantum systems.

Objectives

1. Construct computational basis statevectors.
2. Calculate inner products to examine orthogonality.
3. Form a composite state using a tensor product.

Requirements

Python, NumPy, Qiskit Statevector, Jupyter Notebook.

Theory

Quantum states are vectors in a complex Hilbert space. The inner product measures the overlap between two states; orthogonal basis states have zero overlap. The tensor product combines individual systems into a larger composite state. For example, combining $|0\rangle$ with $|1\rangle$ produces a two-qubit basis state, whose amplitude vector has four entries. These operations are essential for understanding quantum registers and multi-qubit gates.

Key Equation(s)

$\langle a|b\rangle = \Sigma_i \, a_i^* \, b_i$; $\langle 0|1\rangle = 0$; $|0\rangle \otimes |1\rangle = |01\rangle$.

Algorithm / Steps of Execution

1. Import NumPy and Statevector.
2. Create $|0\rangle$ and $|1\rangle$ as separate statevectors.
3. Extract their amplitude arrays.
4. Compute the complex inner product using np.vdot().
5. Form the tensor product using Statevector.tensor().
6. Print and interpret the overlap and composite-state amplitudes.

Aim

Applications

1. Multi-qubit state modeling.
2. Quantum register construction.
3. Entanglement and circuit analysis.

Notebook: EXP-03.ipynb

EXPERIMENT 04

Identity Operators for One, Two and Three Qubits

To construct identity-gate operators and compare their matrix dimensions.

Objectives

1. Define identity operations on qubits.
2. Generate matrices for one, two and three qubits.
3. Verify that identity operations preserve states.

Requirements

Python, Qiskit QuantumCircuit and Operator, NumPy, Jupyter Notebook.

Theory

An identity operator leaves a quantum state unchanged. Its matrix has ones on the principal diagonal and zeros elsewhere. A single-qubit identity is 2 × 2, while two- and three-qubit identities are 4 × 4 and 8 × 8. Applying identity gates to each qubit produces an operator equal to the identity on the full register. Identity operations help verify circuit construction and matrix representations.

Key Equation(s)

$I_2 = [[1,0],[0,1]]$; $I_2^n|\psi\rangle = |\psi\rangle$; $I_2 \otimes I_2 = I_4$.

Algorithm / Steps of Execution

1. Import QuantumCircuit, Operator and NumPy.
2. Create separate circuits containing one, two and three qubits.
3. Apply the identity gate to every qubit in each circuit.
4. Convert each circuit into its operator matrix.

Aim

5. Display the matrices and check their dimensions.
6. Verify that the matrices equal NumPy identity matrices.

Applications

1. Circuit correctness checks.
2. Quantum operator verification.
3. Multi-qubit matrix modeling.

Notebook: EXP-04.ipynb

EXPERIMENT 05

Single-Qubit Gates: Pauli X, Y, Z and Hadamard

To implement and examine fundamental single-qubit quantum gates.

Objectives

1. Create individual one-qubit gate circuits.
2. Inspect the gate matrices and state transformations.
3. Visualize the resulting circuits and states.

Requirements

Python, Qiskit QuantumCircuit and Statevector, Jupyter Notebook.

Theory

Single-qubit gates are unitary transformations of a two-dimensional statevector. The Pauli X gate exchanges $|0\rangle$ and $|1\rangle$; Pauli Y performs a bit-and-phase transformation; Pauli Z changes the relative phase of $|1\rangle$; and Hadamard transforms basis states into equal superpositions. Comparing output statevectors reveals phase effects that may not be visible from computational-basis measurement probabilities alone.

Key Equation(s)

$X = [[0,1],[1,0]]$; $Z = [[1,0],[0,-1]]$; $Y = [[0,-i],[i,0]]$; $H = (1/\sqrt{2})[[1,1],[1,-1]]$.

Algorithm / Steps of Execution

1. Import QuantumCircuit and Statevector.
2. Prepare a fresh one-qubit circuit initialized to $|0\rangle$ for each gate.

Aim

3. Apply X, Y, Z or H to the target qubit in separate circuits.
4. Draw each circuit and calculate its output statevector.
5. Repeat relevant gates on |1⟩ to observe phase or bit-flip behavior.
6. Compare outputs with the expected matrix transformations.

Applications

1. Quantum algorithm building blocks.
2. Superposition and phase manipulation.
3. Quantum logic gate verification.

Notebook: EXP-05.ipynb

EXPERIMENT 06

Two-Qubit Operations: CNOT, Phase and SWAP

To implement controlled and exchange operations on two-qubit states.

Objectives

1. Demonstrate the conditional action of CNOT.
2. Apply a phase gate to one qubit of a register.
3. Verify the exchange of states using SWAP.

Requirements

Python, NumPy, Qiskit QuantumCircuit, Statevector, Jupyter Notebook.

Theory

Two-qubit systems have four computational basis states. CNOT flips its target when the control is 1, while SWAP exchanges the states of two qubits. A phase gate such as S is a single-qubit gate that can be applied within a two-qubit circuit, introducing a relative phase. Selecting input states carefully is necessary to observe these transformations: for instance, SWAP changes |01⟩ into |10⟩ but leaves |00⟩ unchanged.

Key Equation(s)

CNOT|10⟩ = |11⟩ (control first); SWAP|01⟩ = |10⟩; S = [[1,0],[0,i]].

Aim

Algorithm / Steps of Execution

1. Create a two-qubit circuit and prepare a basis input with control set to 1.
2. Apply CNOT and inspect the output state.
3. Create a fresh circuit with a suitable state for observing an S-gate phase.
4. Apply the S gate to the selected qubit and inspect amplitudes.
5. Prepare |01⟩, apply SWAP, and inspect the resulting state.
6. Compare all results with the expected gate behavior.

Applications

1. Entanglement circuit construction.
2. Quantum data routing.
3. Controlled logic and phase manipulation.

Notebook: EXP-06.ipynb

EXPERIMENT 07

Three-Qubit Fredkin (Controlled-SWAP) Gate

To implement and verify the controlled-SWAP operation on three qubits.

Objectives

1. Identify the control and two target qubits.
2. Construct a Fredkin gate circuit.
3. Verify its behavior for control values 0 and 1.

Requirements

Python, Qiskit QuantumCircuit, Qiskit Aer simulator, Jupyter Notebook.

Theory

The Fredkin gate, also called controlled-SWAP or CSWAP, acts on three qubits. When the control qubit is |0⟩, the target qubits remain unchanged; when the control is |1⟩, their states are exchanged. It is a reversible quantum logic operation. A useful test compares the output of target states |01⟩ and |10⟩ under both control settings. Measurement and simulation make the conditional behavior observable.

Aim

Key Equation(s)

CSWAP|0,a,b⟩ = |0,a,b⟩; CSWAP|1,a,b⟩ = |1,b,a⟩.

Algorithm / Steps of Execution

1. Import QuantumCircuit and a compatible simulator.
2. Create a three-qubit circuit and assign control and target positions.
3. Prepare distinguishable target basis states.
4. Apply cswap(control, target1, target2).
5. Measure the qubits and run the circuit with a fixed number of shots.
6. Repeat with the opposite control value and compare the measured outputs.

Applications

1. Reversible computation.
2. Quantum conditional data exchange.
3. Controlled routing in quantum circuits.

Notebook: EXP-07.ipynb

EXPERIMENT 08

Quantum Circuit Formation: Entanglement and Teleportation

To construct a Bell-pair circuit and examine the stages of quantum teleportation.

Objectives

1. Create entanglement with Hadamard and CNOT gates.
2. Implement Bell-basis measurements.
3. Apply conditional corrections to reconstruct a quantum state.

Requirements

Python, Qiskit QuantumCircuit, simulator, Jupyter Notebook.

Theory

Quantum teleportation transfers an unknown qubit state using a shared entangled pair and two classical measurement bits; it does not physically transmit the original particle or enable faster-than-light communication. A Bell pair is

Aim

prepared with Hadamard and CNOT gates. The sender entangles the input with one member of the pair, measures two qubits and communicates the outcomes. The receiver then applies X and Z corrections conditioned on those outcomes. A separate two-qubit Bell-state circuit provides a simple introduction to circuit formation.

Key Equation(s)

$|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$; $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$; $|\psi\text{out}\rangle = Z^b X^a |\psi\text{pre-correction}\rangle$ (corrections depend on measurement bits).

Algorithm / Steps of Execution

1. Create a two-qubit Bell circuit using H followed by CNOT.
2. Measure or inspect the Bell-state output to confirm correlated results.
3. Create a three-qubit circuit and prepare the state to teleport.
4. Entangle the other two qubits using H and CNOT.
5. Apply the sender's CNOT and H, then measure the sender qubits.
6. Apply classically conditioned X and Z corrections and verify the receiver state.

Applications

1. Quantum communication protocols.
2. Entanglement-assisted information transfer.
3. Quantum network education.

Notebook: EXP-08.ipynb